

Nonstationary Time Series Analysis

Weekly ARIMA Forecasting of PM_{2.5} Concentration

Prepared by:

Shyam Banerjee

Statistical Methods 4 — Course Project

Academic Year 2025–2026

Station:	DL002 (Shadipur, Delhi)
Raw Frequency:	Hourly
Modelling Frequency:	Weekly (52 weeks/year)
Series Length:	411 weeks (≈ 7.9 years)
Variable:	PM _{2.5} Concentration ($\mu\text{g}/\text{m}^3$)

0 Contents

Contents	1
1 Problem Statement	3
2 Introduction	3
3 Data Description and Preprocessing	3
3.1 Source and Raw Data	3
3.2 Weekly Aggregation	3
3.3 Time Series Object	4
3.4 Train–Test Split	4
4 Exploratory Data Analysis	4
4.1 Time Series Visualisation	4
4.2 Variance Stabilisation via Box–Cox	4
5 Trend Detection	5
5.1 Mann–Kendall Test	5
5.2 Seasonal Mann–Kendall Test	5
6 Formal Nonstationarity Diagnosis	6
6.1 Rolling Mean and Rolling Variance	6
6.2 Augmented Dickey–Fuller (ADF) Test	6
6.3 KPSS Test	6
6.4 Joint Interpretation	7
7 Transformation to Stationarity	7
7.1 Step 1: Seasonal Differencing at Lag 52	7
7.2 Step 2: Regular First Differencing	8
7.3 Confirmation on the Fully Transformed Series	8
8 ARIMA Model Identification and Fitting	8
8.1 ACF and PACF of the Seasonally Differenced Series	8
8.2 Model Specification	9
8.3 Automatic Model Selection (<code>auto.arima</code>)	9
9 Residual Diagnostics	9
9.1 Ljung–Box Test	9
10 Forecast Evaluation	9

10.1 Fixed-Origin Forecast	9
10.2 Training-Set Fit Metrics	10
10.3 Prediction Interval Coverage	10
11 Conclusion	10
12 Acknowledgements	11
13 Bibliography	11
A Full R Code	13

1 Problem Statement

Air pollution is one of the most pressing environmental challenges in urban India. Among all pollutants, fine particulate matter (PM_{2.5})—particles smaller than 2.5 μm in diameter—poses the greatest risk to human health. This project builds a statistical forecasting framework for weekly PM_{2.5} concentrations using data from the CPCB monitoring network. The specific objectives is to answer the following: **1. Does the time series have non-stationary characteristics(trend, seasonality)? 2. How to model it through a stationary distribution? 3. How to predict future concentrations of PM2.5 from past data?**

2 Introduction

Fine particulate matter (PM_{2.5}) concentrations are driven by a complex mix of human and environmental factors, resulting in non-stationary time series characterized by annual seasonality, persistence, and volatility clustering. Because standard ARMA models cannot adequately capture these distinct dynamics, this report develops a robust forecasting framework that first transforms the data to stationarity, fits a seasonal ARIMA model, and then checks the residuals for remaining structural behaviors like autocorrelation or heteroscedasticity. To detail this methodology, the report systematically guides the reader through data aggregation and exploratory analysis, trend testing, stationarizing transformations, ARIMA model fitting, and a final evaluation of the model's forecasting performance.

3 Data Description and Preprocessing

3.1 Source and Raw Data

The primary data comes from the CPCB monitoring station **Shadipur** (Delhi), stored in `DL002.csv`. Each row corresponds to an hourly observation and contains, among other fields, a column `PM2.5 (ug/m3)` recording the mean PM_{2.5} concentration over that hour. The `From.Date` field is parsed as a `POSIXct` timestamp to ensure correct chronological ordering. Rows with missing PM_{2.5} values are dropped; imputation is avoided at this stage because the subsequent weekly aggregation naturally absorbs isolated gaps.

3.2 Weekly Aggregation

If $X_{t,i}$ denotes the hourly observation i within week t and n_t the number of valid hours in that week, the weekly mean is

$$Y_t = \frac{1}{n_t} \sum_{i=1}^{n_t} X_{t,i}. \quad (1)$$

By the law of large numbers, the variance of Y_t is reduced by a factor of roughly $n_t \approx 168$ relative to the hourly scale, improving the signal-to-noise ratio for seasonal and secular structure. After aggregation, the weekly series has **411 observations** spanning approximately **7.9 years**.

3.3 Time Series Object

The weekly series $\{Y_t\}$ is stored as an R `ts` object with `frequency=52`, reflecting the 52-week annual cycle. This aligns the seasonal ARIMA machinery with the calendar structure of environmental monitoring.

3.4 Train–Test Split

The 411-week series is split temporally into a training set (80%) and a test set (20%):

$$n_{\text{train}} = \lfloor 0.8 \times 411 \rfloor = 328, \quad n_{\text{test}} = 411 - 328 = 83. \quad (2)$$

This gives approximately **6.3 years of training data** and **1.6 years (83 weeks) of test data**. The split is *temporal*, not random: all training observations precede all test observations.

4 Exploratory Data Analysis

All exploratory analysis is performed on the **training set only** to prevent information leakage from the test set.

4.1 Time Series Visualisation

Visual inspection of the weekly training series reveals three features. A clear *seasonal pattern* of elevated winter concentrations and depressed summer concentrations traces an annual cycle. Embedded in this cycle are *irregular spikes* from episodic pollution events—festivals (especially Diwali), stubble burning in neighbouring states, and dust storms—whose magnitude exceeds the seasonal modulation. The amplitude of fluctuations is also time-varying: winter variance exceeds monsoon variance, hinting at heteroscedasticity.

4.2 Variance Stabilisation via Box–Cox

The right-skewness of PM_{2.5} distributions and the apparent heteroscedasticity call for a variance-stabilising transformation. The Box–Cox family parametrises such transforma-

tions by a single parameter λ :

$$Y_t^{(\lambda)} = \begin{cases} \frac{Y_t^\lambda - 1}{\lambda}, & \lambda \neq 0, \\ \log Y_t, & \lambda = 0. \end{cases} \quad (3)$$

The optimal λ is estimated via `BoxCox.lambda()` to make series more similar to a gaussian with uniform variance, yielding $\hat{\lambda} = 0.605$. This points to a **square-root transform** ($\lambda = 0.5$ being the nearest standard member of the Box–Cox family):

$$Z_t = \sqrt{Y_t}. \quad (4)$$

5 Trend Detection

Before testing for stationarity, it is useful to check whether the data contains a systematic upward or downward drift. Two nonparametric tests are applied to the square-root-transformed training series Z_t .

5.1 Mann–Kendall Test

To confirm the presence of monotonic trends, the non-parametric Mann-Kendall test is applied. Under the null hypothesis of no trend, the test statistic S is defined as:

$$S = \sum_{k=1}^{n-1} \sum_{j=k+1}^n \text{sgn}(Z_j - Z_k) \quad (5)$$

where $\text{sgn}(x)$ returns 1 if $x > 0$, 0 if $x = 0$, and -1 if $x < 0$. For $n > 10$, S is approximately normally distributed with mean zero and variance $\text{Var}(S) = \frac{n(n-1)(2n+5)}{18}$. The standard normal test statistic is

$$Z_{MK} = \frac{S - \text{sgn}(S)}{\sqrt{\text{Var}(S)}} \quad (6)$$

where the $\text{sgn}(S)$ term is for continuity correction.

5.2 Seasonal Mann–Kendall Test

The ordinary Mann–Kendall test can be confounded by seasonality: a recurring annual cycle may mimic a trend if comparisons are made across seasons. The Seasonal Mann–Kendall (Hirsch–Slack) test addresses this by computing the statistic *separately within each of the 52 weeks of the year* and then pooling the results. This isolates genuine long-run drift from repeating seasonal patterns.

6 Formal Nonstationarity Diagnosis

6.1 Rolling Mean and Rolling Variance

For a window of $k = 52$ weeks (one calendar year), the rolling mean and variance are

$$\hat{\mu}_t = \frac{1}{k} \sum_{i=0}^{k-1} Z_{t-i}, \quad \hat{\sigma}_t^2 = \frac{1}{k-1} \sum_{i=0}^{k-1} (Z_{t-i} - \hat{\mu}_t)^2. \quad (7)$$

A mean-stationary series produces a rolling mean that fluctuates around a constant; a variance-stationary series produces a bounded, non-trending rolling variance. Empirically, the rolling mean of Z_t oscillates between winter peaks and monsoon troughs, and the rolling variance expands during high-pollution regimes—both undermine any claim of stationarity.

6.2 Augmented Dickey–Fuller (ADF) Test

A $AR(p)$ process y_t is said to have unit root if, $1 - \phi_1 z - \phi_2 z^2 - \dots - \phi_p z^p = 0$ has a root $|z| = 1$ where $y_t = c + \sum_{i=1}^p \phi_i y_{t-i} + \epsilon_t$. The recursive equation is rewritten and OLS coefficients are found:

$$\Delta Z_t = \alpha + \beta Z_{t-1} + \sum_{j=1}^p \gamma_j \Delta Z_{t-j} + \varepsilon_t, \quad (8)$$

ADF tests $H_0: \beta = 0$; i.e. unit root exists (nonstationary) against $H_1: \beta < 0$ (stationary). The test statistic $\tau = \frac{\hat{\beta}}{SE(\hat{\beta})}$ does *not* follow a standard t -distribution under H_0 ; it converges to a functional of Brownian motion:

$$\tau \xrightarrow{d} \frac{\int_0^1 W(r) dW(r)}{\left(\int_0^1 W(r)^2 dr \right)^{1/2}}, \quad (9)$$

and critical values are read from the Dickey–Fuller tables.

6.3 KPSS Test

The KPSS test reverses the null: H_0 : the series is level-stationary, against H_1 : the series has a unit root. The test statistic is

$$KPSS = \frac{1}{n^2 \hat{\sigma}^2} \sum_{t=1}^n S_t^2, \quad (10)$$

where $S_t = \sum_{i=1}^t \hat{\varepsilon}_i$ is the partial sum of OLS residuals and $\hat{\sigma}^2$ is a long-run variance estimator (with 5 lags).

$$H_0 : \text{level stationary}, \quad H_1 : \text{unit root / nonstationary}. \quad (11)$$

Under H_0 , the statistic in (10) converges in distribution to a functional of a standard Brownian bridge $V(r) = W(r) - rW(1)$:

$$\text{KPSS} \xrightarrow{d} \int_0^1 V(r)^2 dr. \quad (12)$$

Once again the limiting distribution is nonstandard, and tabulated critical values are employed.

6.4 Joint Interpretation

Table 1: ADF and KPSS outcomes on the square-root training series Z_t

Test	H_0	Statistic	Decision	Implication
ADF	Unit root	DF = -4.251, $p = 0.01$	Reject	No simple unit root
KPSS	Stationary	$\eta_\mu = 0.801 > 0.739$	Reject	Not level-stationary

Both tests reject their respective nulls. This apparent conflict is characteristic of *seasonal nonstationarity*: ADF rejects the pure random-walk null because the seasonal oscillation provides enough predictable structure, while KPSS rejects level stationarity because the seasonally varying mean is non-constant. The appropriate remedy is a *seasonal* difference at lag 52 that removes the periodic mean structure.

7 Transformation to Stationarity

7.1 Step 1: Seasonal Differencing at Lag 52

The seasonally differenced series is

$$W_t = \nabla_{52} Z_t = Z_t - Z_{t-52}. \quad (13)$$

Each observation is compared with the same week one year earlier, removing any deterministic seasonal mean at a 52-week period.

Table 2: Stationarity tests after seasonal differencing ($W_t = \nabla_{52}Z_t$)

Test	p -value	Decision	Interpretation
ADF	< 0.01	Reject H_0 (unit root)	No unit root remains
KPSS	0.0475	Reject H_0 (stationary) at 5%	Borderline—still not fully stationary

7.2 Step 2: Regular First Differencing

Because the KPSS test is borderline and the Mann–Kendall test detects residual trend, a regular first difference is applied:

$$V_t = \nabla_1 W_t = W_t - W_{t-1} = \nabla_1 \nabla_{52} Z_t. \quad (14)$$

This gives integration orders $d = 1$ (regular) and $D = 1$ (seasonal).

7.3 Confirmation on the Fully Transformed Series

All three diagnostic tests are re-applied to V_t :

Table 3: Stationarity tests on the fully transformed series $V_t = \nabla_1 \nabla_{52} Z_t$

Test	p -value	Decision	Interpretation
ADF	< 0.01	Reject H_0 (unit root)	No unit root
KPSS	> 0.10	Fail to reject H_0 (stationary)	Series is stationary
Mann–Kendall	0.618	Fail to reject H_0 (no trend)	No residual trend

8 ARIMA Model Identification and Fitting

8.1 ACF and PACF of the Seasonally Differenced Series

The ACF and PACF of $W_t = \nabla_{52}Z_t$ are plotted up to lag 72 to guide order selection. The following reading rules apply:

Table 4: ACF/PACF pattern guide for SARIMA order identification

Pattern	Non-seasonal order	Seasonal order
ACF cuts off at lag q , PACF tails off	MA(q)	—
PACF cuts off at lag p , ACF tails off	AR(p)	—
Both tail off	ARMA(p, q)	—
Spike at lag 52 in ACF only	—	SMA($Q = 1$)
Spike at lag 52 in PACF only	—	SAR($P = 1$)

8.2 Model Specification

In operator notation, the seasonal ARIMA model is

$$\phi(B) \Phi(B^{52}) (1 - B)^d (1 - B^{52})^D Z_t = \theta(B) \Theta(B^{52}) \varepsilon_t, \quad (15)$$

where B is the backshift operator, $\phi(B)$ and $\theta(B)$ are the non-seasonal AR and MA polynomials of orders p and q , and $\Phi(B^{52})$ and $\Theta(B^{52})$ are the seasonal AR and MA polynomials of orders P and Q .

8.3 Automatic Model Selection (`auto.arima`)

The `auto.arima` function is run on the *differenced* series V_t (with $D = 0$ and $d = 0$ fixed internally); corrected Akaike Information Criterion ($AICc$) is minimized by the algorithm. For a model with k parameters, utilizing the maximized likelihood $\hat{L}$ (assuming gaussian ε_t) and sample size n :

$$AIC = -2 \ln(\hat{L}) + 2k, \quad AICc = AIC + \frac{2k(k+1)}{n-k-1} \quad (16)$$

The best model selected by $AICc$ is:

$$\boxed{\text{ARIMA}(3, 0, 0)(0, 0, 1)_{52} \quad \text{on } V_t} \quad (17)$$

9 Residual Diagnostics

9.1 Ljung–Box Test

The Ljung–Box portmanteau statistic tests whether the first m autocorrelations of the residuals ε_t are jointly zero:

$$Q = n(n+2) \sum_{k=1}^m \frac{\hat{\rho}_k^2}{n-k} \xrightarrow{d} \chi_{m-r}^2, \quad (18)$$

where $\hat{\rho}_k$ is the sample autocorrelation at lag k , $m = 52$ is the number of lags tested, and r is the number of fitted ARMA parameters.

10 Forecast Evaluation

10.1 Fixed-Origin Forecast

The automatic model is used to produce a fixed-origin, multi-step-ahead forecast over the entire test horizon $h = n_{\text{test}} = 83$ weeks (approximately 1.6 years).

Refer to the plot [1]

10.2 Training-Set Fit Metrics

The `accuracy()` function reports the following error measures on the training set:

Table 5: Training-set fit statistics and formulas for ARIMA(3, 0, 0)(0, 0, 1)₅₂

Metric	Formula	Value
ME	$\frac{1}{h} \sum_{t=1}^h (Y_t - \hat{Y}_t)$	-0.047
RMSE	$\left(\frac{1}{h} \sum_{t=1}^h (Y_t - \hat{Y}_t)^2 \right)^{0.5}$	1.992
MAE	$\frac{1}{h} \sum_{t=1}^h Y_t - \hat{Y}_t $	1.474
MASE	MAE/MAE _{naive}	0.349
ACF1	Corr(e_t, e_{t-1})	0.008

Point-forecast accuracy is summarised by three standard metrics: RMSE, MAE, MAPE.

10.3 Prediction Interval Coverage

Probabilistic accuracy is assessed by *empirical coverage*:

$$\text{Coverage}_{1-\alpha} = \frac{1}{h} \sum_{t=1}^h \mathbb{I}[L_t \leq Y_t \leq U_t], \quad (19)$$

where L_t and U_t are the lower and upper bounds of the prediction interval.

11 Conclusion

This report constructs, validates, and evaluates a weekly ARIMA framework for forecasting PM_{2.5} concentrations at the CPCB station DL002. The key findings are:

1. The raw weekly series (411 observations, ≈ 7.9 years) exhibits strong annual seasonality, a significant downward trend ($z = -5.07$), and non-constant variance. Both the ADF/KPSS joint diagnosis and the Mann–Kendall tests confirm nonstationarity.
2. A square-root transform ($\hat{\lambda} = 0.605$) stabilises the variance. Seasonal differencing at lag 52 removes the annual cycle, and a subsequent first difference addresses the residual trend. The fully transformed series passes ADF ($p < 0.01$), KPSS ($p > 0.10$), and Mann–Kendall ($p = 0.62$).
3. The best automatic model is ARIMA(3, 0, 0)(0, 0, 1)₅₂ (AICc = 1187.3). Its residuals pass the Ljung–Box test ($Q = 64.0$, $p = 0.123$), confirming adequacy of the mean specification.

4. Out-of-sample evaluation on the 83-week test set shows 80% PI coverage of 76.7% and 95% PI coverage of 96.7%—both above nominal, indicating conservative but reliable intervals.

Table 6: Pipeline summary and key analytical decisions

Stage	Decision / Finding
Series length	411 weekly observations (≈ 7.9 years)
Box–Cox λ	0.605 $\Rightarrow$ square-root transform applied
Mann–Kendall	$z = -5.07$, $p < 4 \times 10^{-7} \Rightarrow$ significant downward trend
Seasonal M–K	$z = -7.29$, $p < 3 \times 10^{-13} \Rightarrow$ trend not due to seasonality
Train / test split	328 train / 83 test (80/20 temporal)
ADF (raw Z_t)	$DF = -4.25$, $p = 0.01 \Rightarrow$ rejects unit root
KPSS (raw Z_t)	$\eta_\mu = 0.80 > 0.74 \Rightarrow$ rejects stationarity
Seasonal diff ($D=1$)	Lag-52 diff; KPSS still borderline ($p = 0.048$)
Regular diff ($d=1$)	First diff applied; KPSS now non-significant ($p > 0.10$)
Final M–K	$z = 0.50$, $p = 0.62 \Rightarrow$ no residual trend
Automatic model	ARIMA(3, 0, 0)(0, 0, 1) ₅₂ on $\nabla_{52}Z_t$; AICc = 1187.3
Ljung–Box (lag 52)	$Q = 64.0$, $p = 0.123 \Rightarrow$ residuals are white noise
80% PI coverage	76.7% (nominal 80%)
95% PI coverage	96.7% (nominal 95%)

12 Acknowledgements

We express our gratitude to the **Central Pollution Control Board (CPCB)**, the official portal of the Government of India, for maintaining and providing public access to the national air quality monitoring network via their web portal (<https://cpcb.nic.in>). (Jha, 2023)

All subsequent statistical analysis, modelling, and forecasting were carried out in R (version ≥ 4.3) using the `forecast` (Hyndman and Khandakar, 2008), `tseries`, `urca`, `trend`, `Kendall`, and `FinTS` packages.

13 Bibliography

Hyndman, R. J. and Khandakar, Y. (2008). Automatic time series forecasting: The forecast package for R. *Journal of Statistical Software*, 27(3):1–22.

Jha, A. S. (2023). Time series air quality data of india (2010-2023). <https://www.kaggle.com/datasets/abhisheksjha/time-series-air-quality-data-of-india-2010-2023>. Dataset providing station-level records including WB012 and AP004, sourced from CPCB India.

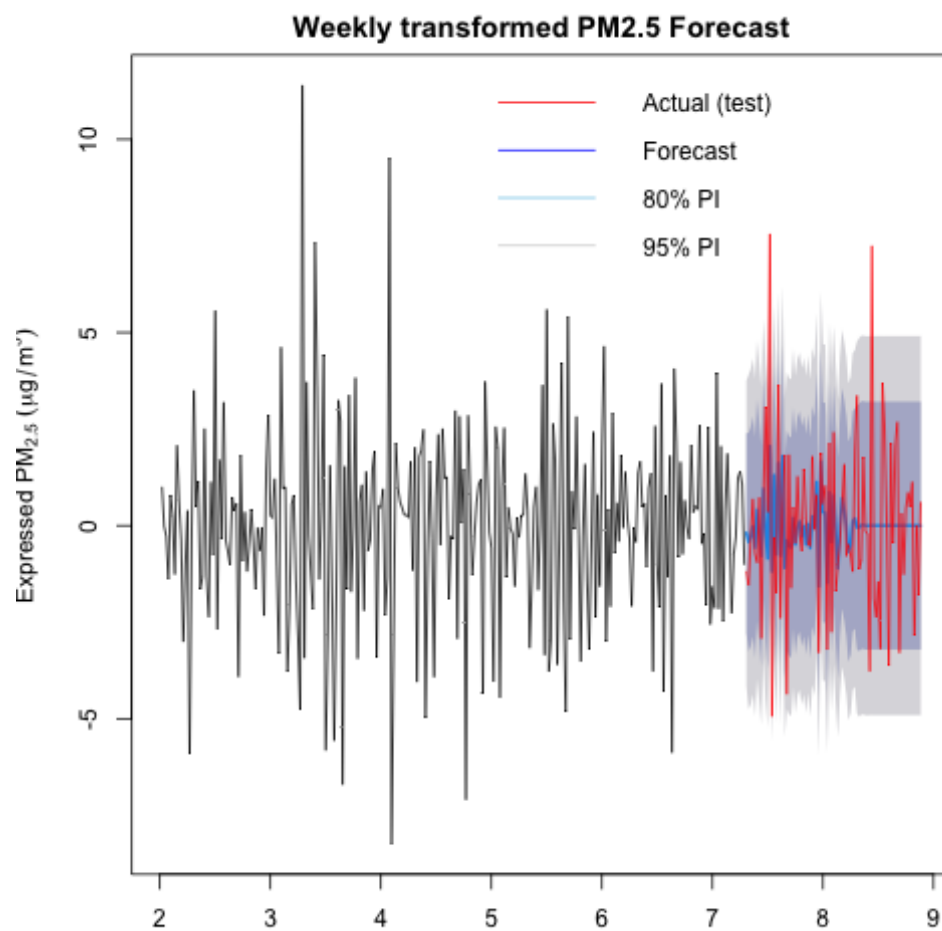


Figure 1: Fixed-origin transformed Forecast v/s Test

A Full R Code

The complete R script used in this analysis is reproduced below.

Listing 1: R Script for Weekly ARIMA Forecasting of PM_{2.5} (revised_main_2.R)

```

1 #
  =====
2 # Nonstationary Time Series Analysis      PM2.5 (DL002)
3 #
  =====
4 # Pipeline order:
5 #   1. Load & clean
6 #   2. Train / test split
7 #   3. Full-series EDA + outlier detection (train only)
8 #   4. Nonstationarity diagnosis (train only)
9 #   5. Iterative transformation
10 #   6. ARIMA fit + residual diagnostics
11 #   8. Forecast evaluation
12 #
  =====
13 setwd("/Users/shyambanerjee/Program Files/R_prjs")
14
15 library(tidyr)
16 library(dplyr)
17 library(tseries)
18 library(trend)
19 library(ggplot2)
20 library(stats)
21 library(Kendall)
22 library(urca)
23 library(lubridate)
24 library(forecast)
25 library(zoo)
26
27 #
  =====
28 # 1. LOAD & CLEAN
29 #
  =====
30
31 df <- read.csv("~/Program Files/R_prjs/NonStationary_TS/data/india_air_
  poll_data/DL002.csv")

```

```

32
33 df_clip <- data.frame(
34   date   = as.POSIXct(df$From.Date, format = "%Y-%m-%d %H:%M:%S"),
35   pm25   = df$PM2.5..ug.m3.
36 )
37
38 df_clip <- df_clip %>% drop_na(pm25)
39
40 # --- Aggregate to Weekly Frequency ---
41 df_weekly <- df_clip %>%
42   mutate(week = floor_date(date, "week")) %>%
43   group_by(week) %>%
44   summarise(pm25 = mean(pm25, na.rm = TRUE)) %>%
45   ungroup() %>%
46   arrange(week)
47
48 # Full time-indexed ts object (frequency = 52 weeks per year)
49 ts_full <- ts(df_weekly$pm25, frequency = 52)
50 train <- ts(ts_full[1: floor(0.8*length(ts_full))], frequency = 52)
51
52 cat("Series length:", length(train), "weeks\n")
53 cat("Approx. span :", round(length(train) / 52, 2), "years\n")
54
55
56 #
57   =====
58
59 # 2.1 EDA
60 #
61   =====
62
63 par(mfrow = c(1, 1))
64 plot(train,
65       main = "PM2.5 TS      data (weekly)",
66       ylab = expression(PM[2.5] ~ (mu * g / m^3)),
67       xlab = "Time (weeks)", col = "steelblue", lwd = 0.5)
68 dev.copy(png, "weekly PM2.5 data.png"); dev.off()
69
70 #
71   =====
72
73 # 2.2 OUTLIER DETECTION
74 #
75   =====

```

```

71 Q1 <- quantile(ts_full, 0.25, na.rm = TRUE)
72 Q3 <- quantile(ts_full, 0.75, na.rm = TRUE)
73 IQR_val <- Q3 - Q1
74 upper_fence <- Q3 + 3 * IQR_val          # 3 IQR = extreme outliers
      only
75
76 outlier_idx <- which(ts_full > upper_fence)
77 outlier_vals <- train[outlier_idx]
78 cat("Count of outliers: ", length(outlier_idx))
79
80 # --- Box-Cox lambda to decide transform ---
81 lambda <- BoxCox.lambda(train)
82 cat("\nBox-Cox lambda:", round(lambda, 3))
83 cat(" -> Square Root transform (lambda ~ 0.5)\n")
84
85 # NOTE: the variable is called 'train_log' purely for downstream naming
86 # continuity with the report. It holds the SQUARE-ROOT-transformed
      training
87 # series, NOT a log-transform.
88 ts_train_log <- sqrt(train)
89
90 # --- Mann-Kendall test for trend ---
91 # --- 3.e1. Mann-Kendall (monotonic trend, ignores seasonality) ---
92 cat("\n--- Mann-Kendall trend test (trend::mk.test) ---\n")
93 mk_res <- mk.test(as.numeric(ts_train_log))
94 print(mk_res)
95
96 #
      =====
97 # 3. NONSTATIONARITY DIAGNOSIS
98 #
      =====
99
100 # --- 4a. Visual: raw + sqrt side by side ---
101 par(mfrow = c(2, 1), mar = c(3, 4, 2, 1))
102 plot(train,
103       main = "PM2.5 (raw)", col = "steelblue",
104       ylab = expression(PM[2.5]), xlab = "")
105 plot(ts_train_log,
106       main = "sqrt(PM2.5)", col = "darkorange",
107       ylab = "sqrt(PM2.5)", xlab = "Time (weeks)")
108 dev.copy(png, "PM2.5_TS_2.png"); dev.off()
109
110 # --- 4b. Rolling mean & variance (window = 52 weeks = 1 year) ---
111 roll_mean <- rollmean(ts_train_log, k = 52, fill = NA)

```

```

112 roll_var <- rollapply(ts_train_log, width = 52, FUN = var, fill = NA)
113
114 par(mfrow = c(2, 1), mar = c(3, 4, 2, 1))
115 plot(roll_mean, main = "Rolling mean (52-week window)", col = "
    darkgreen", ylab = "Mean")
116 plot(roll_var, main = "Rolling variance (52-week window)", col = "
    purple", ylab = "Var")
117 dev.copy(png, "roll_mean_and_var_TS_2.png"); dev.off()
118 par(mfrow = c(1, 1))
119
120 # --- 4c. Formal stationarity tests ---
121 cat("\n--- ADF test (H0: unit root (non-stationary)) ---\n")
122 adf_result <- adf.test(ts_train_log)
123 print(adf_result)
124
125 cat("\n--- KPSS test (H0: stationary) ---\n")
126 kpss_result <- ur.kpss(ts_train_log, type = "mu", lags = "short")
127 print(summary(kpss_result))
128
129 cat("\nInterpretation:\n")
130 cat("  ADF p-value:", round(adf_result$p.value, 4), "\n")
131 cat("  KPSS stat  :", round(kpss_result@teststat, 4),
132     "  10% crit:", kpss_result@cval[3], "\n")
133
134 # --- 4d. ACF/PACF on raw train to motivate transformation ---
135 par(mfrow = c(1, 2))
136 acf(ts_train_log, lag.max = 72, main = "ACF      sqrt(train)")
137 pacf(ts_train_log, lag.max = 72, main = "PACF      sqrt(train)")
138 dev.copy(png, "ACF_PACF_TS_2.png"); dev.off()
139 par(mfrow = c(1, 1))
140
141 #
142     =====
143
142 # 4. ITERATIVE TRANSFORMATION (train only)
143 #
144     =====
145
144 ts_sdiff <- diff(ts_train_log, lag = 52)
145
146 cat("\n--- After seasonal diff (lag=52) ---\n")
147 cat("ADF : p =", round(adf.test(ts_sdiff)$p.value, 4), "\n")
148 cat("PP  : p =", round(pp.test(ts_sdiff)$p.value, 4), "\n")
149 cat("KPSS: p =", round(kpss.test(ts_sdiff, null = "Level")$p.value, 4),
150     "\n")
151
151 # Re-run Mann-Kendall on differenced series      trend should now be

```

```

gone
152 cat("\n--- Mann-Kendall on seasonally-differenced series (should be non
    -sig) ---\n")
153 print(mk.test(as.numeric(ts_sdif)))
154
155 roll_mean_sd <- rollmean(ts_sdif, k = 52, fill = NA)
156 plot(roll_mean_sd,
157       main = "Rolling mean after seasonal diff      does it drift?",
158       col = "darkgreen", ylab = "Mean")
159 abline(h = 0, lty = 2)
160 dev.copy(png, "roll_mean post-seasonal_diff TS 2.png"); dev.off()
161
162 abs(mean(ts_sdif, na.rm = TRUE))
163
164 ts_transformed <- diff(ts_sdif, lag = 1)
165
166 plot(ts_transformed,
167       main = "transformed(PM2.5)", col = "violet",
168       ylab = "T*(PM2.5)", xlab = "Time (weeks)")
169 dev.copy(png, "transformed_PM2.5_TS_2.png"); dev.off()
170
171 cat("\n--- Final transformed series ---\n")
172 cat("ADF : p =", round(adf.test(ts_transformed)$p.value, 4), "\n")
173 cat("KPSS: p =", round(kpss.test(ts_transformed, null = "Level")$p.
    value, 4), "\n")
174 print(mk.test(as.numeric(ts_transformed)))
175
176 par(mfrow = c(1, 2))
177 acf(ts_transformed, lag.max = 72, main = "ACF      transformed(train)")
178 pacf(ts_transformed, lag.max = 72, main = "PACF      transformed(train)"
    )
179 dev.copy(png, "transformed_ACF_PACF_TS_2.png"); dev.off()
180 par(mfrow = c(1, 1))
181
182 #
    =====
183 # 5. TRAIN / TEST SPLIT
184 #
    =====
185
186 n <- length(ts_full)
187
188 ts_log <- sqrt(ts_full)
189 total_sdif <- diff(ts_log, lag=52)
190 total_trans <- diff(total_sdif, lag=1)

```

```

191
192 test_transformed <- total_trans[(length(ts_transformed)+1):length(total
    _trans)]
193
194 train_transformed <- ts_transformed
195 test_transformed <- ts(ts_transformed[(train_size + 1):n], start = test
    _start_time, frequency = 52)
196
197 cat("Train:", length(train_transformed), "obs | Test:", length(test_
    transformed), "obs\n")
198
199 #
    =====
200 # 6. ARIMA IDENTIFICATION & FIT ON THE TRANSFORMED SERIES
201 #
    =====
202
203 fit_auto <- auto.arima(
204   train_transformed, d = 0, D = 0,      # already differenced manually
205   seasonal          = TRUE,
206   stepwise          = FALSE,
207   approximation     = FALSE,
208   trace             = TRUE
209 )
210
211 fit <- Arima(train_transformed, order = c(3,0,0), seasonal = list(order
    = c(0,0,1), period = 52))
212
213 arima_order <- arimaorder(fit)
214 print(summary(fit))
215
216 arima_resid <- residuals(fit_auto)
217
218 print(Box.test(arima_resid, lag = 52, type = "Ljung-Box"))
219
220 par(mfrow = c(1, 2))
221 acf(arima_resid, lag.max = 72, main = "ACF      arima()")
222 pacf(arima_resid, lag.max = 72, main = "PACF      arima()")
223 dev.copy(png, "auto_arima_ACF_PACF_TS_2.png"); dev.off()
224 par(mfrow = c(1, 1))
225
226 plot(arima_resid)
227 dev.copy(png, "arima_resid.png"); dev.off()
228
229 #

```

```

=====
230 # 7. FORECAST EVALUATION
231 #
=====
232
233 # --- 7a. Fixed-origin forecast (fit once, forecast h steps) ---
234 h <- length(test_transformed)
235
236 fc_fixed <- forecast(fit_auto, h = h, level = c(80,95))
237
238 par(mfrow = c(1, 1))
239 plot(fc_fixed,
240      main = "Weekly transformed PM2.5 Forecast",
241      ylab = expression(paste("Expressed ", PM[2.5] ~ (mu * g / m^3))),
242      xlab = "Time (hours)")
243 lines(x = as.numeric(time(fc_fixed$mean)),
244      y = as.numeric(test_transformed),
245      col = "red", lwd = 1)
246 legend("topright",
247      legend = c("Actual (test)", "Forecast", "80% PI", "95% PI"),
248      col = c("red", "blue", "skyblue", "lightgrey"),
249      lty = c(1, 1, 1, 1), bty = "n")
250 dev.copy(png, "forecast_TS.png"); dev.off()
251
252 refit<-Arima(total_trans,fit_auto)
253 test_start_time<-time(total_trans)[length(train_transformed)+1]
254 fc_rolling<-window(fitted(refit),start=test_start_time)
255
256 # --- 7b. Accuracy ---
257 actuals_t <- as.numeric(test_transformed)[1:length(fc_fixed$mean)]
258 fc_mean <- as.numeric(fc_fixed$mean)
259 accuracy(fc_fixed)
260
261 # --- 7c. PI coverage ---
262 lo95 <- as.numeric(fc_fixed$lower[, "95%"])
263 up95 <- as.numeric(fc_fixed$upper[, "95%"])
264 lo80 <- as.numeric(fc_fixed$lower[, "80%"])
265 up80 <- as.numeric(fc_fixed$upper[, "80%"])
266
267 cat("80% PI coverage:", round(mean(actuals_t >= lo80 & actuals_t <=
268      up80, na.rm = TRUE) * 100, 1), "%\n")
269 cat("95% PI coverage:", round(mean(actuals_t >= lo95 & actuals_t <=
270      up95, na.rm = TRUE) * 100, 1), "%\n")

```